New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 14 — Módulo 4:

Autenticación Frontend con Angular Standalone



Objetivo del módulo: implementar el inicio de sesión y persistencia de estado de autenticación en Angular 18 Standalone con Bootstrap 5, construyendo la pantalla de login, formularios reactivos y el servicio AuthService con Signals.



New Horizons®
Centro de Capacitación

Agenda

- Arquitectura de autenticación en SPAs: Ciclo de vida del token en el cliente
- Manejo de estado de autenticación reactivo con Angular Signals
- Diseño de la pantalla de Login empresarial con Bootstrap 5 & Bootstrap Icons
- Formulario reactivo de login con validación sincrónica de formato y obligatoriedad
- Consumo del endpoint POST /api/auth/login, almacenamiento seguro y redirección



New Horizons®
Centro de Capacitación

Sesión 1:

Autenticación Frontend en Angular Standalone con Bootstrap 5

Visión general de la sesión

New Horizons®
Centro de Capacitación



El login es la recepción de un edificio corporativo. El backend (Sesión 13) verifica tu identidad y te entrega un fotocheck: el token. Hoy construimos la recepción en Angular: la pantalla, el formulario y el servicio que recuerda quién eres.

Arquitectura Robusta

Componentes standalone y un AuthService que separa la lógica de la vista.

Piénsalo así: *Recepcionista y archivo están en escritorios distintos: puedes cambiar uno sin tocar el otro.*



Alto Desempeño

Signals actualizan solo lo que depende del usuario autenticado.

Piénsalo así: *Como un letrero digital: cambia una línea, no se reimprime todo el edificio.*



Seguridad Total

Validación previa, mensajes genéricos y sesión que se limpia al salir.

Piénsalo así: *El fotocheck se devuelve en la puerta: nada queda en el escritorio de nadie.*

Pilares técnicos fundamentales

New Horizons
Centro de Capacitación

El viaje del fotocheck: cinco paradas que recorre el usuario desde que llega hasta que entra al sistema.

1

Ciclo de vida del token

Arquitectura de autenticación en SPAs: cómo nace, vive y muere el token en el cliente.

2

Estado con Signals

Manejo reactivo del estado de autenticación: toda la app se entera al instante.

3

Login con Bootstrap 5

Pantalla de login empresarial con Bootstrap 5 y Bootstrap Icons.

4

Formulario reactivo

Validación sincrónica de formato y obligatoriedad antes de llamar a la API.

5

POST y redirección

Consumo de POST /api/auth/login, almacenamiento seguro y navegación.

🗺️ Al final de la sesión, cada parada tendrá su código funcionando en el proyecto de **Solicitudes de Gasto**.

Implementación Técnica — Bloque 1

New Horizons
Centro de Capacitación

Sesion14.cs

```
// Implementación de referencia para la Sesión 14
1 public class ConfiguracionSesion14 {
    // Arquitectura de autenticación en SPAs:
    // Ciclo de vida del token en el cliente
    public static void Configurar(IServiceCollection services) {
2       services.AddAuthorization();
    }
}
```

Dos lados del mismo mostrador

Servidor (C#)

Emite y valida el token. Decide quién entra.

Cliente (Angular)

Pide el token, lo guarda y lo muestra. Hoy trabajamos aquí.



Los cimientos del edificio

Antes de abrir la recepción, el edificio necesita reglas de acceso. Ese reglamento vive en el servidor.

1 **Clase de configuración**

Punto único donde el backend registra sus servicios de seguridad.

2 **services.AddAuthorization()**

Registra el motor de políticas que decide qué puede hacer cada usuario.

3 **¿Y el frontend?**

Angular no reemplaza estas reglas: solo mejora la experiencia. La última palabra es del servidor.

Práctica: Pantalla y Formulario Reactivo de Login

New Horizons
Centro de Capacitación

Como un arquitecto: primero la maqueta (tarjeta centrada y estilos), luego la instalación eléctrica (ReactiveFormsModule y validaciones).

1 Paso 1 · Maqueta de la vista

Diseñamos la vista centrada con tarjeta corporativa y logos institucionales:

- ✓ Contenedor `min-vh-100` con `d-flex` para centrar
- ✓ Tarjeta `card shadow-lg` de ancho máximo 420 px
- ✓ Logo e íconos con Bootstrap Icons (`bi-shield-lock`)
- ✓ `ReactiveFormsModule` en los imports del componente

2 Paso 2 · Validación y compilación

Validación y compilación en limpio de los componentes.

- ✓ Campos con `is-invalid` y `invalid-feedback`
- ✓ Botón deshabilitado mientras se envía
- ✓ `ng build` sin errores ni warnings
- ✓ Prueba manual: correo vacío, mal formado y correcto

Ciclo de vida del token en una SPA

New Horizons
Centro de Capacitación

🎟️ **El token es la pulsera de un hotel todo incluido:** te la ponen en recepción, la muestras en cada restaurante, vence en el check-out y, si la pierdes, vuelves a recepción.

🔑 1

Login

El usuario envía correo y clave a POST /api/auth/login.

🎫 2

Emisión

La API responde con el JWT y su fecha de expiración.

🗄️ 3

Guardar

AuthService lo guarda en sessionStorage y actualiza el signal.

✉️ 4

Usar

Viaja en el header Authorization (interceptor, Sesión 15).

⏰ 5

Expira

Pasada la hora, la API responde 401 y hay que volver a entrar.

🚪 6

Logout

Se borra el storage, el signal vuelve a null y se redirige.

🤔 Pregunta guía: **¿qué pasa en cada parada si el usuario recarga la página?** La respuesta está en la lámina de almacenamiento.

¿Dónde guardar el token en el navegador?

New Horizons
Centro de Capacitación

Tres lugares, tres riesgos distintos. No existe la opción perfecta; existe la adecuada para tu contexto.

`localStorage`

La caja fuerte del lobby

✓ Sobrevive al cerrar el navegador
✗ Cualquier script inyectado (XSS) la abre
✗ Compartida entre pestañas

Evitar para tokens

`sessionStorage`

El casillero que se vacía al salir

✓ Aislado por pestaña
✓ Se borra al cerrar la pestaña
✗ También es legible por JavaScript

Nuestra elección en el curso

`Cookie HttpOnly`

El maletero del valet

✓ JavaScript no puede leerla
✗ Requiere protección CSRF
✗ Se configura en el backend (Secure, SameSite)

Ideal en producción

Ninguna opción reemplaza lo básico: **HTTPS, tokens de vida corta y no pintar HTML sin sanear** (Angular ya lo hace por ti).

AuthService: el estado de sesión con Signals

auth.service.ts

```
1  @Injectable({ providedIn: 'root' })
   export class AuthService {
     private readonly CLAVE = 'sesion';

     // Estado privado: solo el servicio puede escribir
2    private _usuario = signal<UsuarioSesion | null>(
       this.leerSesion()
     );

     // Estado público: los componentes solo leen
3    usuario = this._usuario.asReadonly();
4    estaAutenticado = computed(() => this._usuario() !== null);
     nombre = computed(() => this._usuario()?.nombre ?? 'Invitado');
   }
```



El letrero de vuelos

Un solo letrero, cientos de pasajeros mirándolo. Cuando cambia, todos se enteran sin preguntar.

1 providedIn: 'root'

Una sola instancia para toda la app: un único letrero.

2 signal privado

Solo el servicio cambia el estado (login, logout).

3 asReadonly()

Los componentes leen, pero no pueden escribir.

4 computed()

Valores derivados que se recalculan solos.

signal, computed y effect en la práctica

New Horizons®
Centro de Capacitación



Piensa en un termostato inteligente: la temperatura es un signal, "¿hace frío?" es un computed y el aire que se enciende solo es un effect.



signal

La temperatura actual

Valor reactivo que puedes leer y escribir.

```
usuario = signal<Usuario |
null>(null);
usuario.set(resp.usuario);
```



computed

"¿Hace frío?"

Se deriva de otros signals y se recalcula solo.

```
estaAutenticado = computed(
  () => this.usuario() !== null);
```



effect

El aire que se enciende

Ejecuta un efecto lateral cuando algo cambia.

```
effect(() => document.title =
  `Hola, ${this.nombre()}`);
```



Regla práctica: signal para el estado, computed para lo derivado y effect solo para efectos laterales (nunca para escribir otros signals).

Contratos del login: Request y Response

auth.models.ts

```ts
export interface LoginRequest {
  correo: string;
  clave: string;
}

export interface UsuarioSesion {
  id: number;
  nombre: string;
  rol: string;
}

export interface LoginResponse {
  token: string;
  expiraEn: string;          // fecha ISO 8601
  usuario: UsuarioSesion;
}
```



El contrato de alquiler

Inquilino y propietario firman las mismas cláusulas. Si una cambia, ambos deben enterarse.

Así llega la respuesta (200 OK)

```
POST /api/auth/login

{
  "token": "eyJhbGciOi...",
  "expiraEn": "2026-10-26T18:30:00Z",
  "usuario": {
    "id": 7, "nombre": "Ana Torres",
    "rol": "Solicitante"
  }
}
```



Los nombres deben coincidir 1:1 con el DTO del backend de la Sesión 13.

Pantalla de Login con Bootstrap 5

New Horizons
Centro de Capacitación

login.component.html

```
1  <div class="container min-vh-100 d-flex
               align-items-center justify-content-center">
2    <div class="card shadow-lg border-0"
         style="max-width: 420px; width: 100%;">
       <div class="card-body p-5">
         <div class="text-center mb-4">
3          <i class="bi bi-shield-lock-fill text-primary fs-1"></i>
           <h1 class="h4 fw-bold mt-2">Portal de Solicitudes</h1>
         </div>
4        <form [formGroup]="loginForm" (ngSubmit)="ingresar()">
           <!-- campos: ver siguientes láminas -->
         </form>
       </div>
     </div>
   </div>
```

1 **Centrado total**
min-vh-100 + flexbox

2 **Tarjeta**
sombra, sin borde, 420 px

3 **Identidad**
Bootstrap Icons

4 **Formulario**
conectado al FormGroup

La puerta principal

Es la primera impresión del sistema: limpia, centrada y con la marca de la empresa.

Portal de Solicitudes

Correo corporativo

Contraseña

Ingresar

Formulario reactivo de login

New Horizons®
Centro de Capacitación

login.component.ts

```ts
@Component({
  selector: 'app-login',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './login.component.html'
})
export class LoginComponent {
  private fb = inject(FormBuilder);
  private auth = inject(AuthService);
  private router = inject(Router);

  cargando = signal(false);
  error = signal<string | null>(null);

  loginForm = this.fb.nonNullable.group({
    correo: ['', [Validators.required, Validators.email]],
    clave: ['', [Validators.required, Validators.minLength(8)]]
  });
}
```



El control antes de embarcar

Revisa que traigas documento y que sea válido antes de ir a la puerta de embarque (la API).

1 imports: [ReactiveFormsModule]

Standalone: cada componente declara lo que usa.

2 Validators.email

Formato correcto antes de gastar una petición.

3 minLength(8)

Obligatoriedad y longitud mínima de la clave.

4 signals de UI

`cargando` y `error` alimentan el spinner y la alerta.

Validación visual con Bootstrap 5

New Horizons
Centro de Capacitación

login.component.html / .ts

```
<div class="form-floating mb-3">
  <input type="email" class="form-control" id="correo"
         formControlName="correo"
         placeholder="nombre@empresa.com"
1        [class.is-invalid]="campoInvalido('correo')">
  <label for="correo">Correo corporativo</label>
2  <div class="invalid-feedback">Ingresa un correo válido.</div>
</div>

// login.component.ts
campoInvalido(nombre: string): boolean {
  const c = this.loginForm.get(nombre);
3  return !!c && c.invalid && (c.touched || c.dirty);
}
```

El corrector ortográfico

Subraya en rojo solo después de que escribiste la palabra, no mientras la piensas.

Así lo ve el usuario

Correo corporativo

ana.torres@

Ingresa un correo válido.

1 [class.is-invalid]

Activa el borde rojo cuando la regla falla.

2 invalid-feedback

Bootstrap lo muestra solo junto a is-invalid.

3 touched || dirty

La regla de oro: no regañar antes de tiempo.

Consumo de POST /api/auth/login

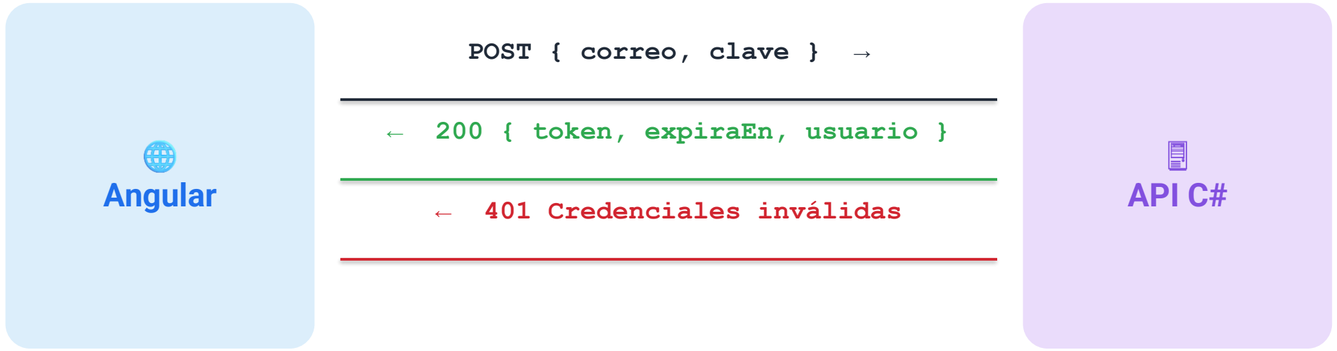
New Horizons
Centro de Capacitación

auth.service.ts

```
// AuthService (continuación)
private http = inject(HttpClient);
private router = inject(Router);
1 private api = environment.apiUrl;

login(datos: LoginRequest): Observable<LoginResponse> {
  return this.http
2   .post<LoginResponse>(`${this.api}/auth/login`, datos)
3   .pipe(tap(resp => this.guardarSesion(resp)));
}
```

La conversación con la API



La ventanilla del banco

Entregas tu formulario, recibes un ticket. `tap` es anotar el ticket en tu libreta sin modificarlo.

1 environment.apiUrl

La URL cambia por ambiente; el código no.

2 post<LoginResponse>

Respuesta tipada con el contrato de la lámina anterior.

3 tap(guardarSesion)

Efecto lateral: guarda la sesión y deja pasar la respuesta intacta.

Guardar y restaurar la sesión

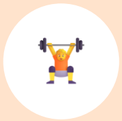
New Horizons
Centro de Capacitación

auth.service.ts

```ts
// AuthService (continuación)
private guardarSesion(resp: LoginResponse): void {
  sessionStorage.setItem(this.CLAVE, JSON.stringify(resp));
  this._usuario.set(resp.usuario);
}

private leerSesion(): UsuarioSesion | null {
  const raw = sessionStorage.getItem(this.CLAVE);
  if (!raw) return null;
  const sesion: LoginResponse = JSON.parse(raw);
  const vigente = new Date(sesion.expiraEn) > new Date();
  return vigente ? sesion.usuario : null;
}
```

1. `sessionStorage.setItem(this.CLAVE, JSON.stringify(resp));`
2. `this._usuario.set(resp.usuario);`
3. `const vigente = new Date(sesion.expiraEn) > new Date();`



El casillero del gimnasio

Guardas tus cosas y anotas tu número en la pizarra. Al volver, abres el casillero y revisas que tu pase no haya vencido.

¿Qué pasa al presionar F5?

🔄 **Recarga la página**

📂 **leerSesion() abre el casillero**

⏰ **¿expiraEn ya pasó?**

No → sesión restaurada

Sí → volver al login

El signal se inicializa con leerSesion(): la app arranca ya sabiendo quién eres.

Envío del formulario y redirección

New Horizons®
Centro de Capacitación

login.component.ts

```ts
ingresar(): void {
  if (this.loginForm.invalid) {
1   this.loginForm.markAllAsTouched();
    return;
  }
  this.cargando.set(true);
  this.error.set(null);

  this.auth.login(this.loginForm.getRawValue()).subscribe({
2   next: () => this.router.navigate(['/solicitudes']),
    error: (e: HttpErrorResponse) => {
3     this.error.set(e.status === 401
        ? 'Correo o contraseña incorrectos.'
        : 'No pudimos conectar. Intenta nuevamente.');
      this.cargando.set(false);
    }
  });
}
```

El mostrador de check-in

Si tus documentos están en regla, te envían a la puerta de embarque. Si no, te lo dicen con claridad.

✅	**200 OK**	navigate(['/solicitudes'])
⛔	**401**	“Correo o contraseña incorrectos.”
⚠️	**0 / 500**	“No pudimos conectar…”

🙊 **Mensaje genérico:** nunca digas si falló el correo o la clave; eso le da pistas a un atacante.

Feedback de carga y errores en la UI

New Horizons
Centro de Capacitación

login.component.html

```
@if (error()) {
  <div class="alert alert-danger d-flex align-items-center">
    <i class="bi bi-exclamation-triangle-fill me-2"></i>
    {{ error() }}
  </div>
}

<button type="submit" class="btn btn-primary w-100 py-2"
        [disabled]="cargando()">
  @if (cargando()) {
    <span class="spinner-border spinner-border-sm me-2"></span>
    Validando...
  } @else {
    <i class="bi bi-box-arrow-in-right me-2"></i> Ingresar
  }
</button>
```

1. Muestra el error del servicio.
2. Deshabilita el botón mientras valida.
3. Cambia el texto según el estado.



El indicador del ascensor

Nadie presiona el botón diez veces si ve que el ascensor ya viene en camino.

Los tres estados del botón

→ Ingresar — `listo`

◌ Validando... — `cargando()`

⚠ Correo o contraseña incorrectos.

💡 `[disabled]="cargando()"` evita el doble clic y el doble envío.

Cierre de sesión seguro

New Horizons
Centro de Capacitación

auth.service.ts / navbar.component.html

```
// AuthService (continuación)
logout(): void {
  sessionStorage.removeItem(this.CLAVE);
  this._usuario.set(null);
  this.router.navigate(['/login'], { replaceUrl: true });
}

<!-- navbar.component.html -->
@if (auth.estaAutenticado()) {
  <span class="navbar-text me-3">{{ auth.nombre() }}</span>
  <button class="btn btn-outline-light btn-sm"
          (click)="auth.logout()">Salir</button>
}
```

Devolver el fotocheck

Al salir del edificio entregas el fotocheck, se borra tu registro y la puerta se cierra detrás de ti.

1. **removeItem()**

   Borra el token del casillero (sessionStorage).

2. **_usuario.set(null)**

   Todos los computed reaccionan: la navbar se oculta sola.

3. **replaceUrl: true**

   El botón “atrás” no regresa a la pantalla protegida.

Errores comunes y cómo evitarlos

New Horizons®
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Omitir la validación estricta de parámetros en el servidor	Validar siempre los contratos DTO en cada llamada HTTP
Almacenar información sensible en repositorios públicos	Utilizar variables de entorno seguras en desarrollo y producción
Desacoplar la UI de las políticas de seguridad del backend	Mantener arquitectura standalone y tipado estricto
Decir “el usuario no existe” o “clave incorrecta” por separado	Un mensaje genérico ante cualquier 401
Leer sessionStorage en cada componente	Centralizar el estado en AuthService con Signals

Buenas prácticas de seguridad en el login

New Horizons
Centro de Capacitación

🚲 **Un candado de bicicleta no sirve si dejas la llave puesta:** la seguridad del login está en los detalles pequeños.

🙈 Mensajes genéricos

Ante un 401 no reveles si falló el correo o la clave.

“Correo o contraseña incorrectos.”

🔒 HTTPS siempre

Credenciales y token solo viajan cifrados, también en QA.

environment.apiUrl con https://

⏳ Tokens de vida corta

Respeta expiraEn y limpia la sesión al vencer.

leerSesion() descarta sesiones vencidas

🧹 Nada sensible a la vista

Ni console.log(token) ni la clave guardada en ningún storage.

Solo el token, nunca la contraseña

Resumen de la Sesión 14

New Horizons®
Centro de Capacitación

Estado con Signals

Angular Signals permite un estado de autenticación inmutable, reactivo y de alto rendimiento.

≈ El letrero de vuelos

Pantalla Bootstrap 5

Experiencia visual limpia y responsive para el acceso corporativo.

≈ La puerta principal

sessionStorage

Aísla la sesión por pestaña y previene fugas en equipos compartidos.

≈ El casillero

Formulario reactivo

La validación en el cliente evita peticiones con datos incompletos.

≈ El control antes de embarcar

Desacoplamiento

AuthService separado de la vista: pruebas unitarias sencillas y limpias.

≈ Recepción y archivo

Full Stack

Login de punta a punta, listo para producción.

≈ El fotocheck completo

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Dónde es más seguro almacenar el token JWT: localStorage, sessionStorage o cookies?
2. ¿Cómo notificamos en tiempo real a toda la aplicación cuando el usuario inicia sesión?
3. ¿Cómo validamos reactivamente que el usuario ingrese un correo válido antes de enviar?
4. ¿Qué ocurre si el usuario recarga la página y cómo restauramos la sesión activa?
5. ¿Cómo implementamos el cierre de sesión seguro limpiando el estado global?

New Horizons
Centro de Capacitación

Conclusiones



Estado reactivo

Angular Signals permite un estado de autenticación inmutable, reactivo y de alto rendimiento.



Acceso corporativo

Bootstrap 5 ofrece una experiencia visual limpia y responsive para la pantalla de acceso corporativo.



Sesión aislada

sessionStorage aísla la sesión por pestaña y previene fugas de credenciales en equipos compartidos.



Filtro previo

La validación sincrónica en el cliente optimiza el tráfico evitando peticiones con datos incompletos.



Código testeable

El desacoplamiento entre AuthService y la vista asegura pruebas unitarias sencillas y limpias.

GRACIAS